



Efficient Inference Serving Systems

Cornserve: Efficiently Serving Any-to-Any Multimodal Models

NIRVANA: Approximate Caching for Efficiently Serving Diffusion Models

Zhengqing Liu, Barry Lyu, Yile Sun, Jingjing Zhang



Cornserve

Efficiently Serving Any-to-Any Multimodal Models



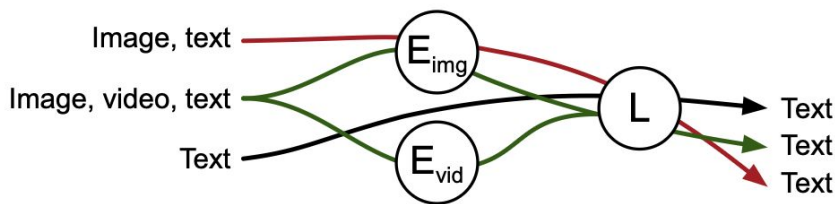
Motivation: Challenges in GenAI Serving

Optimizing Text-Only LLMs $\neq$ Optimizing Multi-Modal GenAI

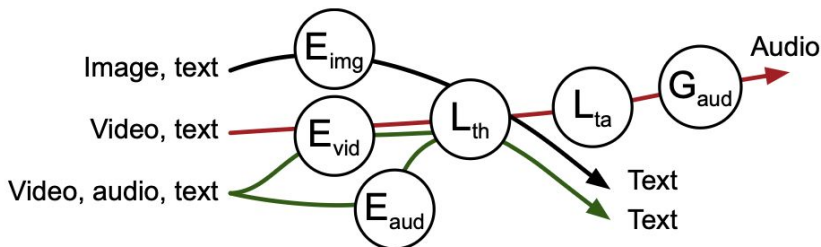
- Heterogeneous workload (6-40% text-only workload)
- Heterogeneous model component (Multi-Modal Encoders + Generators)
- Optimization levers depends model and workload characteristics



Background: Any-to-Any Multimodal Models



(a) *Multimodal input, text output (MLLM)*



(b) *Multimodal input, multimodal output*

E: encoder
L: LLM
G: generator
 L_{th} : thinker
 L_{ta} : talker



Challenge

1. Request type and computation path heterogeneity
2. Computation scaling heterogeneity

	Image input	Video input	Audio input	Text in/out	Audio output	Image output
Qwen 2.5 Omni	15.64	1.28	34.04	1.09	0.28	
Qwen 2.5 VL	12.04	0.89		0.32		
InternVL 3	1.13	0.74		0.28		
Qwen-Image				15.67		0.20

Maximum throughput (requests/s) of each modality component on one A100-80GB. Empty cells mean the model does not have that component.



4 Monoliths	4.4	3.5	2.8	3.2	4.4	2.5	2.4	4.2
2E/3PD	3.2	3.2	3.2	3.8	4.0	2.6	2.8	4.3
2EP/2D	2.0	2.2	2.1	2.5	2.6	2.5	1.5	2.7
2E/1P/2D	2.4	2.4	1.8	2.1	3.0	2.1	1.6	2.6
	1920x1080x1 input 100 output 100	1920x1080x1 input 100 output 300	1920x1080x1 input 1000 output 300	1680x1050x1 input 1000 output 300	1680x1050x1 input 100 output 300	1680x1050x1 input 1000 output 500	896x896x2 input 1000 output 300	896x896x1 input 1000 output 300

E: encoder
P: prefill
D: decode

The best deployment strategy depends on the model and the workload.



Existing Systems are Point Solutions

- vLLM or SGLang: text-only and multimodal input LLMs
- xDiT: diffusion models

Need an automatic planner for any-to-any model



System Overview

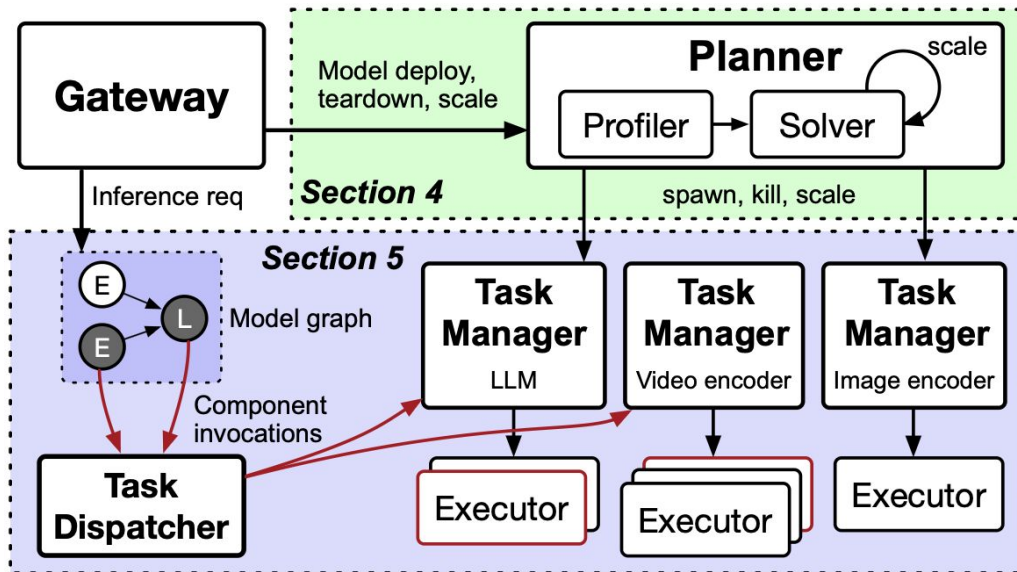
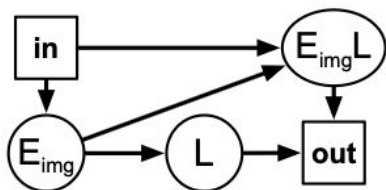


Figure 3: Cornserve architecture.



Planner: Inputs & Outputs

Model executor graph



Workload
(request distribution)



Planner

Deployment plan:

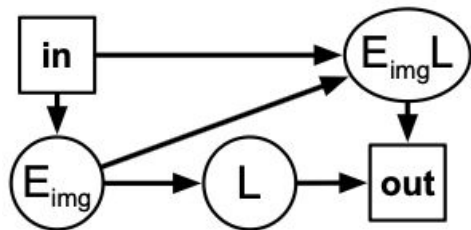
- Mixture of executor options
- Number of replicas for each option

Objective:

1. Maximize request throughput given a GPU budget.
2. Minimize GPU usage given a target request throughput.

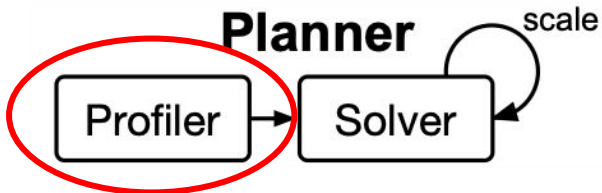


Model executor graph



Workload
(request distribution)

Planner

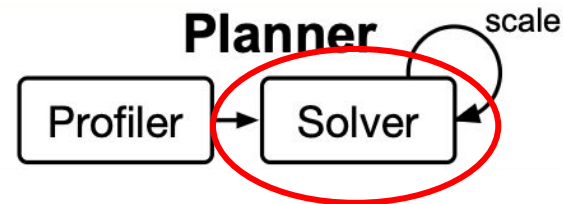


Throughput per request type GPUs per replica

	Image	Text	Image & text	
--	-------	------	--------------	--

	Image	Text	Image & text	
E_{img}	10.0			1
L		3.33		2
$E_{img}L$		3.03	2.27	2

Profiling results



Multicommodity network design problem

- Goal: find **minimal number of GPUs** given target request throughput
- Variables:
 - Placement: How many replicas of executor s run on node h ?
 - Routing: How much traffic of type t takes path p ?
 - Physical Flow: How much data flows between node h and node h' for logical connection (s, s') ?
- Constraints:
 - Demand constraint, node capacity, link capacity

NP Hard



Planner: Cell Abstraction

- Insight:
 - Intra-server: communication is cheap
 - Inter-server: communication is expensive
- **Cell**: fundamental unit of high-bandwidth compute (always a power of 2)



Planner: Intra-Cell Planning

Simplified multicommodity network design problem

- Goal: find **minimal number of GPUs** given target request throughput
- Variables:
 - Placement: How many replicas of executor s ~~run on node h ?~~
 - Routing: How much traffic of type t takes path p ?
 - ~~Physical Flow: How much data flows between node h and node h' for logical connection (s, s') ?~~
- Constraints:
 - Demand constraint, node capacity, ~~link capacity~~



Planner: Building Efficient Cells

Input: Maximum cell exponent $c_{\max}$ (max size $2^{c_{\max}}$)

Output: Efficient cells $\mathbf{C} = \{(g[c], R[c])\}$

▷ Multicommodity network design (§4.2)

1 **for** $c \leftarrow 0$ **to** $c_{\max}$ **do**

→ For each possible power of 2 size

2 $g[c] \leftarrow 2^c$

3 $R[c] \leftarrow \text{BudgetMNDP}(g[c])$

→ Run the intra-cell solver

▷ Filter inefficient cells

4 $\mathbf{C} \leftarrow \{(g_0, R_0)\}$

5 $c^* \leftarrow 0$

▷ Index of largest efficient cell so far

6 **for** $c \leftarrow 1$ **to** $c_{\max}$ **do**

7 **if** $R_c > 2^{c-c^*} R_{c^*}$ **then**

8 $\mathbf{C} \leftarrow \mathbf{C} \cup \{(g_c, R_c)\}$

9 $c^* \leftarrow c$

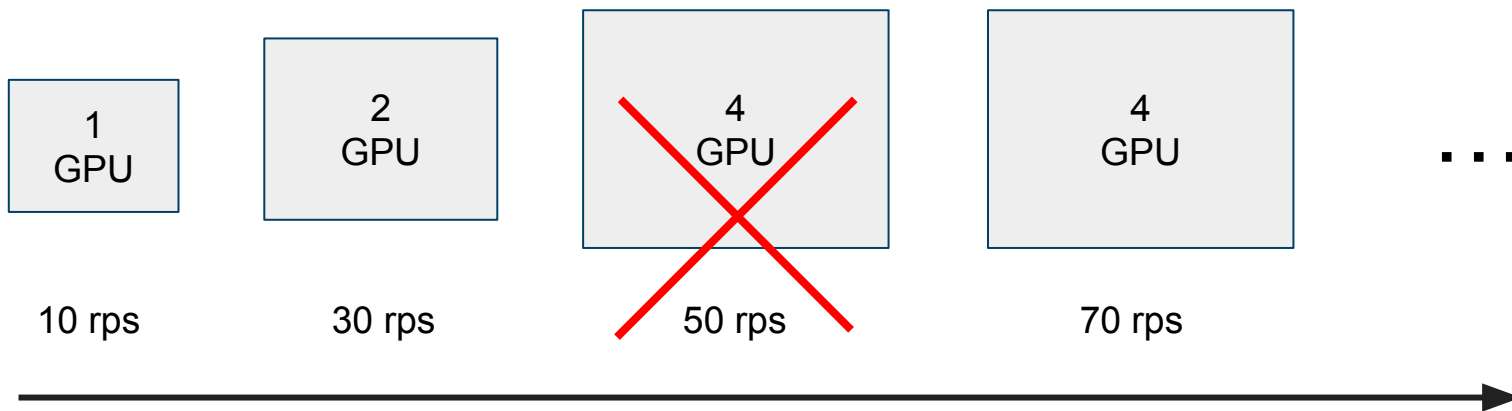
→ Prune inefficient cells:
Make sure a larger cell is better
than the sum of its parts

10 **return** $\mathbf{C}$

Algorithm 1: Planning efficient power-of-two cells.



Planner: Building Efficient Cells



Incrementally search for the optimal intra-cell deployment.



Planner: Offline Inter-Cell Planning

- Input: efficient cells (larger cells are always more efficient)
- Output: cell mixture

Scenario A: maximize throughput (fixed budget N)

Algorithm: binary decomposition

Scenario B: minimize GPUs (fixed demand R)

Algorithm: Greedy

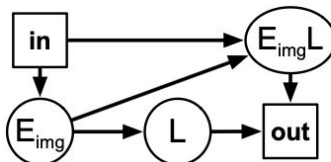


Planner: A Toy Example

Planner Inputs

Workload (request distribution)
Text-only : Image & text = 2 : 9

Two request types



Model definition with paths



	Throughput per request type			GPUs per replica
	Image	Text	Image & text	
E_{img}	10.0			1
L		3.33		2
$E_{img}L$		3.03	2.27	2

Profiling results

Case 1

11 total GPUs
All three executors used

Text-only requests (2 rps)

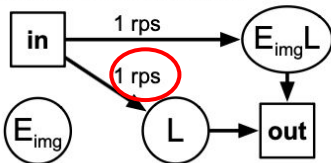
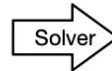
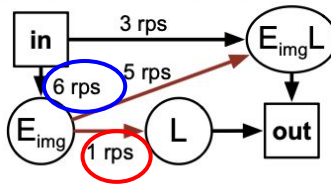


Image & text requests (9 rps)



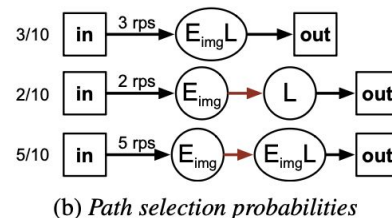
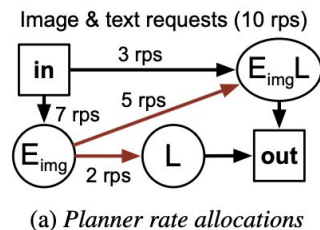
	Work demand	Replicas (round up)	GPUs
E _{img}	6 / 10.0 = 0.60	1	1
L	(1 + 1) / 3.33 = 0.60	1	2
E _{img} L	(1 + 5) / 3.03 + 3 / 2.27 = 3.30	4	8
			= 11

Distributed Runtime

Given an *offline* resource allocation:

Runtime for the *online* serving phase:

- API abstraction to express Any-to-Any model computation.
- Heterogeneous model component (Multi-Modal Encoders + Generators)
- Request routing and load balancing





Task Abstraction

Expressing Any-to-Any Compute:

- Model invocation depends on incoming request type
- Model inference logic can contain arbitrary processing

Provide a library of *unit task* classes:

LLMTask, EncoderTask, ...



MLLM Task

```
1 from cornserve.task.base import Task, Request, Response
2 from cornserve.tasklib import LLMTask, EncoderTask
3
4 class MLLM(Task[Request, Response]):
5     model_id: str
6     disag_enc: bool
7     disag_llm: bool
8
9     # Initialization
10     def post_init(self) -> None:
11         if (not self.disag_enc and self.disag_llm):
12             raise ValueError("Invalid Configuration")
13         # L or EL
14         self.llm = LLMTask(
15             from_model_id=self.model_id,
16             with_enc=not self.disag_llm,
17         )
18         if self.disag_enc:
19             self.enc = EncoderTask(from_model_id=self.model_id)
20
21     # Computation
22     def invoke(self, request: Request) -> Response:
23         # Encoder only invoked if there are images
24         if self.disag_enc:
25             # E -> L or E -> EL
26             embeddings = []
27             for image in request.images:
28                 embeddings.append(self.enc.invoke(image))
29             return self.llm.invoke(
30                 text=request.text,
31                 image_embeddings=embeddings,
32             )
33         else:
34             # EL
35             return self.llm.invoke(request)
```

Whether there is a separate image encoder

Whether LLM has its own encoder

Initialization ->

Computation ->

Router Composite Task

```
1 from cornserve.task.base import Task, Request, Response
2 from cornserve.tasklib import DisagMLLM, MLLM, HybridMLLM
3 from cornserve.tasklib.helpers import replayable_choice
4
5 class MLLMRouter(Task[Request, Response]):
6     paths: list[Task[Request, Response]]
7     # request type probabilities will be the planner output
8     text_only_probabilities: list[float]
9     image_text_probabilities: list[float]
10
11     def invoke(self, request: Request) -> Response:
12         probs = (
13             self.text_only_probabilities
14             if not len(request.images)
15             else self.image_text_probabilities
16         )
17         # Random choice but repeats same choice for replay
18         idx = replayable_choice(len(self.paths), p=probs)
19         path = self.paths[idx]
20         return path.invoke(request)
21
22 model_id = "Qwen/Qwen2.5-VL-7B-Instruct"
23 MLLM(encoder_disag: bool, encoder_coloc: bool)
24 E_L = MLLM(disag_enc=True, disag_llm=True)
25 EL = MLLM(disag_enc=False, disag_llm=False)
26 E_EL = MLLM(disag_enc=False, disag_llm=False)
27 my_router = MLLMRouter(paths=[E_L, E_L, E_EL])
```

Path Probabilities for Text-only vs image+text ->

Select a path ->

Three Possible Paths ->

Task Abstraction Allows Composing Dynamic Tasks

Record & Replay Graph Invocation

Upon receiving request at gateway:

Run `invoke` in *record* mode:

Mock results and record invocations

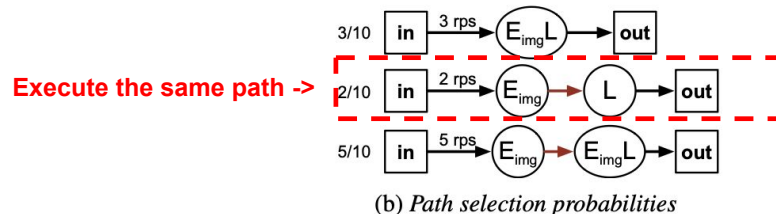
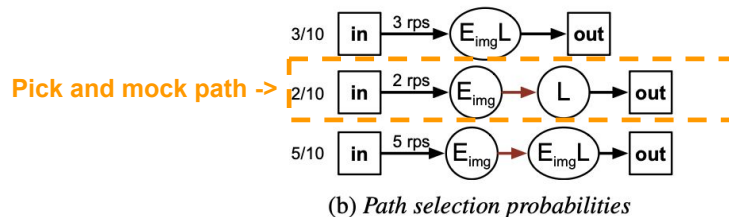
Reconstruct recorded graph and send to Task Dispatcher

Task Dispatcher determines executors for each unit invocation.

Then run `invoke` in *replay* mode:

Preserve same path selection

Run actual computation





Implementation

15000 line of python on top of Kubernetes
LLM executor w/ vLLM

Intra-node transfer:

Shared Memory

Inter-node transfer:

RDMA via UCX

Evaluation Setup

2 nodes, each with 8 A100 GPU and NVLink.
Cross-node bandwidth over RDMA

Text/Audio output: Qwen 3 Omni, 2,5 Omni

Text output: Qwen 2.5 VL 32, Intern VL 3 38B

Image output: Qwen-Image

Baseline:

Text output with vLLM

Image/Audio Output with HuggingFace diffusers +
transformers

Evaluation (Text + Audio Output)

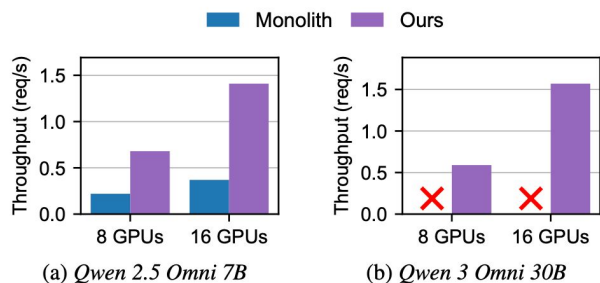


Figure 6: Throughputs of serving Qwen 2.5 Omni [44] and Qwen 3 Omni [45] on different cell sizes. “Cross” means the monolithic deployment does not run due to OOM.

- (a) 3.09x / 3.81x speedup on 8/16 GPU
- (b) 2.68x speedup on 16 vs 8 GPU

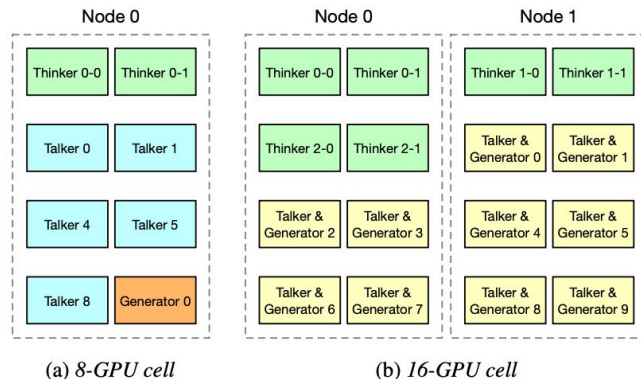
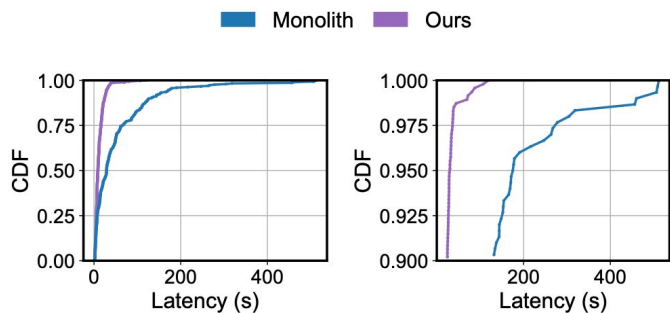


Figure 7: The optimal deployment options of serving Qwen 3 Omni [45] on different cell sizes decided by Cornserve’s planner, corresponding to the two columns in Figure 6b respectively.

More than 2x speed from 8->16 GPU:
8 GPU cell can’t balance throughput

Evaluation (Text + Audio Output)



(a) End-to-end Latency CDF (b) 90th-100th percentile Zoom-in

Figure 8: Qwen 2.5 Omni end-to-end request latency CDF.

Significant improvement in latency,
especially, tail latency.

Input	Output	Monolith			Ours		
		P50	P90	P99	P50	P90	P99
T + I + V + A	T	33.6	128.9	181.4	7.1	18.4	19.8
T + I + V	T	20.8	88.8	150.4	7.3	18.7	22.6
T + I + A	T	20.1	114.7	128.6	7.9	19.1	24.8
T + I	T	28.9	95.8	125.2	8.5	16.3	21.0
T + I + V + A	A	26.9	156.9	167.0	24.8	36.5	37.8
T + I + V	A	88.8	236.5	340.5	21.9	36.7	44.3
T + I + A	A	29.6	182.5	284.5	23.1	33.6	37.7
T + I	A	86.2	288.6	375.2	21.3	30.4	39.6

Table 2: Qwen 2.5 Omni latency statistics by request types on 16 GPUs. I stands for image, V for video, A for audio, and T for text.

- Monolithic models only make progress on one component at a time (CornServe reduces blocking)
- Disaggregating audio generator allows continuous batching on the generator.



Evaluation (Text-only Output w/ Real Workload)

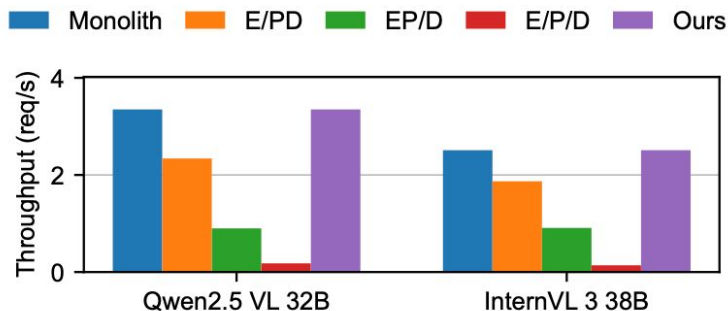


Figure 9: Throughput of serving Qwen 2.5 VL 32B [10] and InternVL3 [52] with the ServeGen workload in a 16 GPU cell.

Monolithic is best: ServeGen workload is decode bound and KV Cache capacity is the bottleneck

Disaggregating increases throughput balance but decreases absolute throughput because it reduces KV Cache.



Evaluation (Text-only Output w/ Controlled Workload)

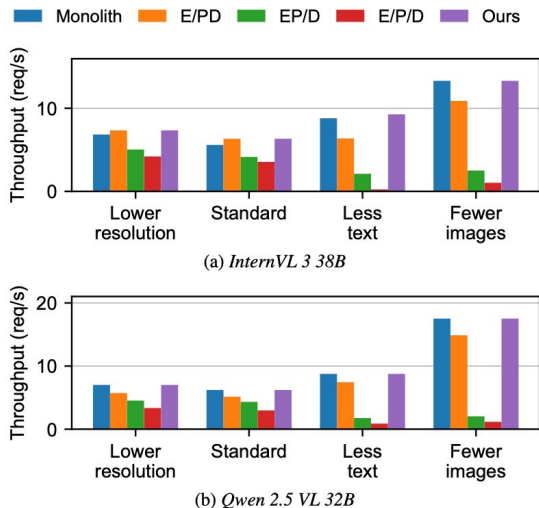
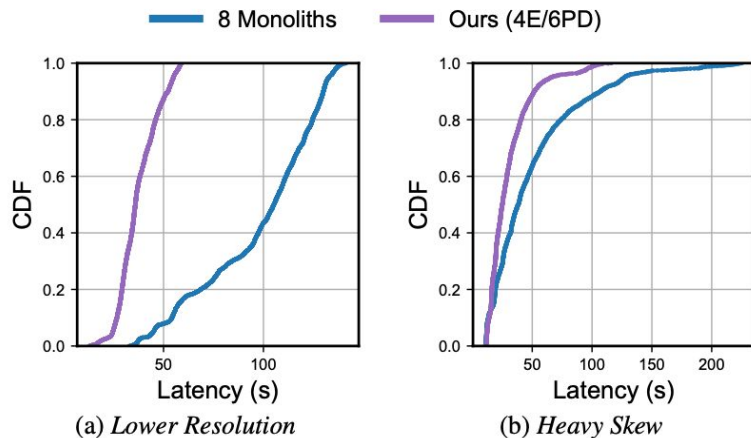


Figure 10: Serving throughput of InternVL 3 38B [52] and Qwen 2.5 VL 32B [10] on 16 GPUs with controlled workloads defined in Table 3 using different deployment strategies.

InternVL 3 38B Less Text: CornServe outperforms existing strategies. Due to (1) disaggregation opportunities (KV Cache is not bottleneck), and (2) better load balancing for encoding.

Qwen 2.5 VL 32B: Monolithic is the best for all workloads. Because the encoder is much smaller, and GPUs may be underutilized. (Also less KV Cache)

Evaluation (Text-only Output w/ Controlled Workload)



CornServe gives superior latency curves:

- (a) Monolithic suffers from limited KV cache and queueing delay
- (b) CornServe reduces E/P/D interference.

Figure 11: End-to-end request latency CDF of serving InternVL 3 38B [52] on 16 GPUs for both solutions (8·2 vs. 4·1 + 6·2) under controlled workloads.



Evaluation (Image Output)

Qwen 2.5 VL 7B + 20B Diffusion Transformer

CornServe always chooses monolithic: Computational heterogeneity between LLM and DiT can't be balanced within 16-GPU cell.

Evaluation (Request-Static Routing)

InternVL 3 38B w/ 4 Encoder + 6 Monolithic deployment.

3.7x higher throughput with request-static routing.

Dynamic Routing:

High Control Plane Overhead: Frequent Queue length queries



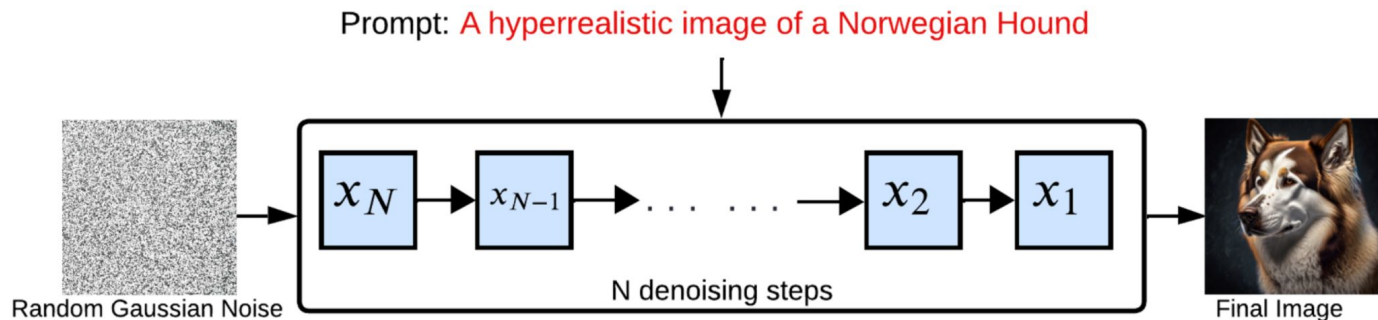
NIRVANA

Approximate Caching for Efficiently Serving Diffusion Models



Diffusion Model

Diffusion models are networks aiming to denoise a random Gaussian noise to generate images conditioned on texts.

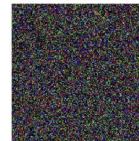
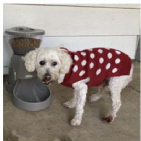
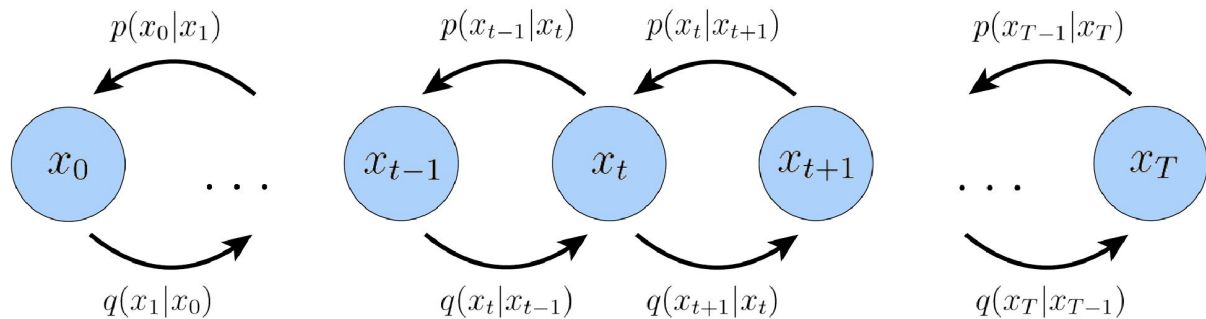


Training of Diffusion Model

Diffusion models consist of two processes:

Forward diffusion process that gradually adds noise to input.

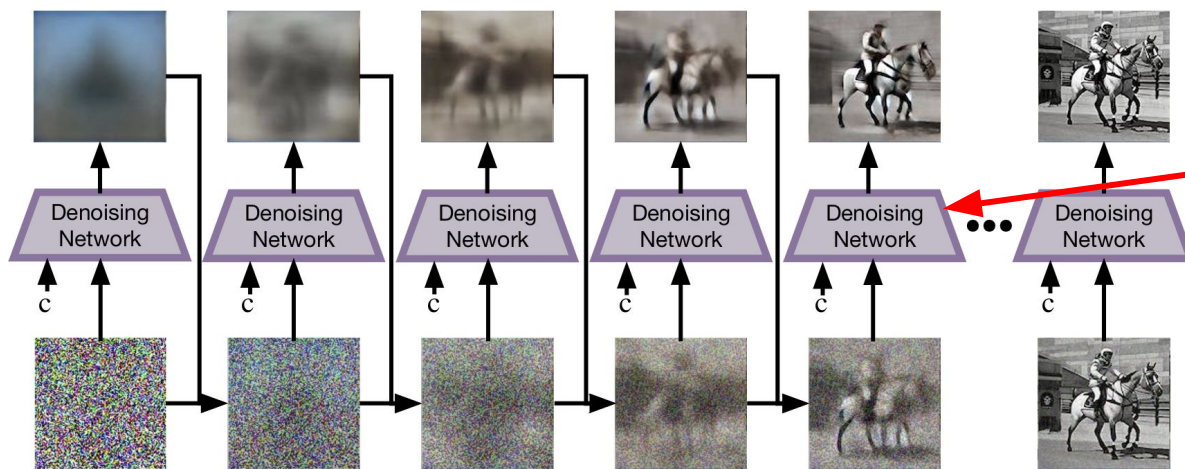
Reverse denoising process that learns to generate data by denoising.





Inference of Diffusion Model

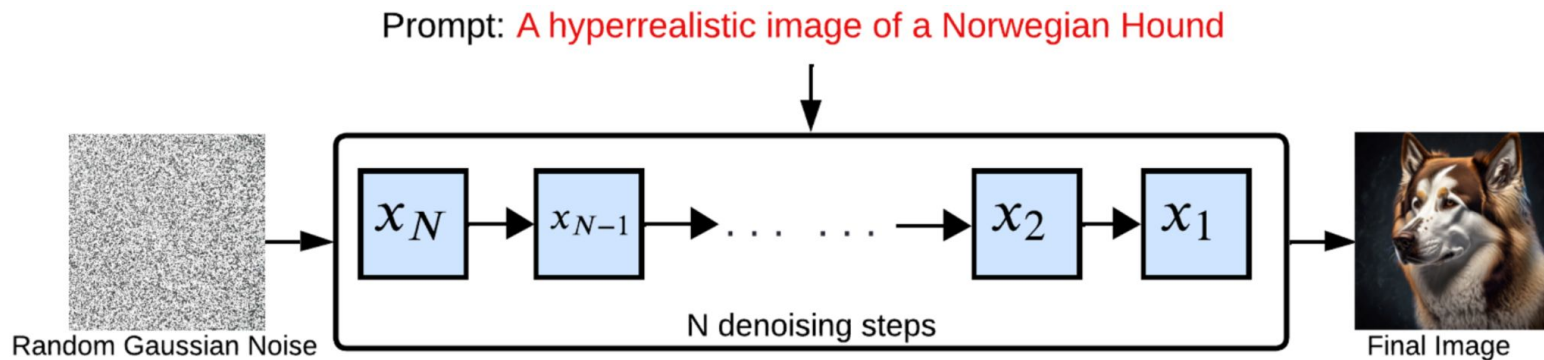
Input Prompt (c): An astronaut riding a horse.



Massive Computation Per Step: Each step requires a full forward pass through a large backbone.



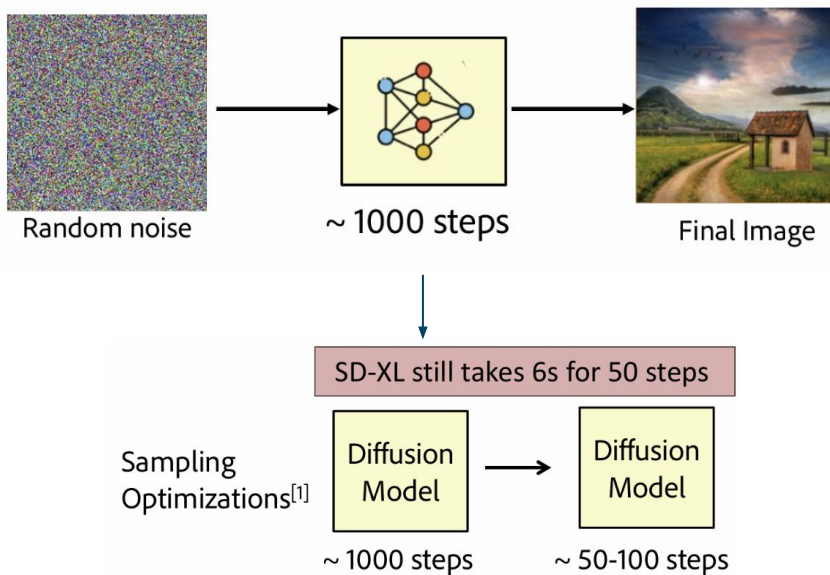
Challenges



Pretty Slow: Traditional diffusion models require ~1000 steps to generate a single image.



Previous Works



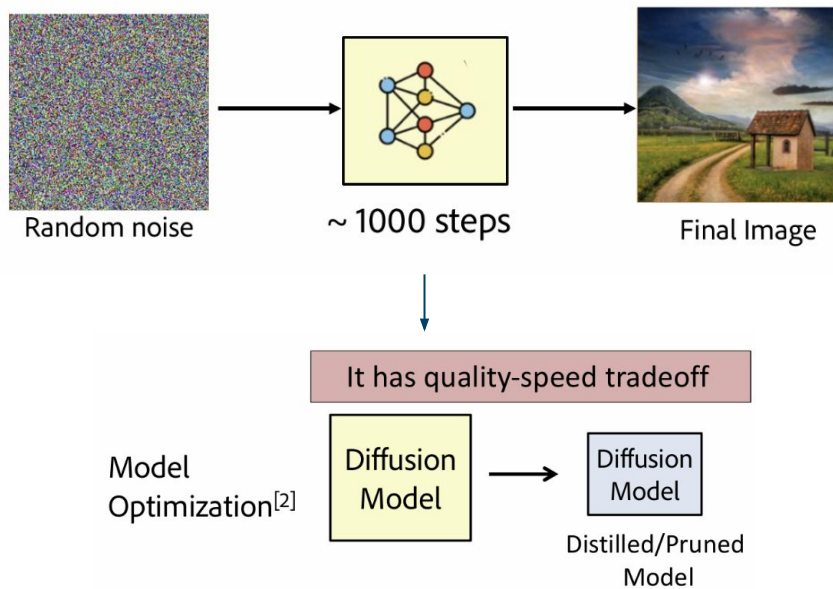
Sampling Optimization:
learn the mathematical
trajectory of the diffusion
process to compress
multiple denoising steps
into one

[1] Hongkai Zheng, Weili Nie, Arash Vahdat, Kamyar Azizzadenesheli, and Anima Anandkumar. Fast sampling of diffusion models via operator learning. In ICML 2023.

[2] Chenlin Meng, Robin Rombach, Ruiqi Gao, Diederik Kingma, Stefano Ermon, Jonathan Ho, and Tim Salimans. On distillation of guided diffusion models. In CVPR, 2023.



Previous Works



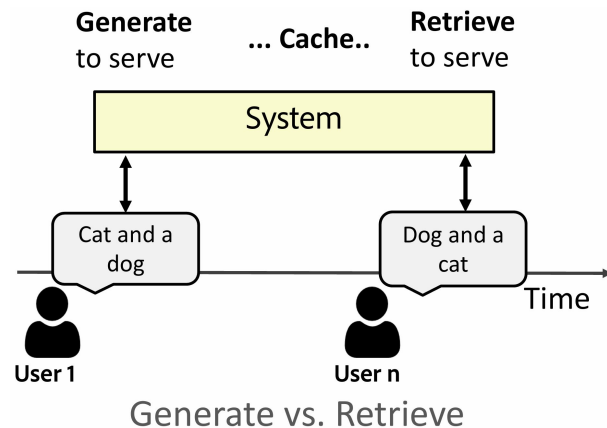
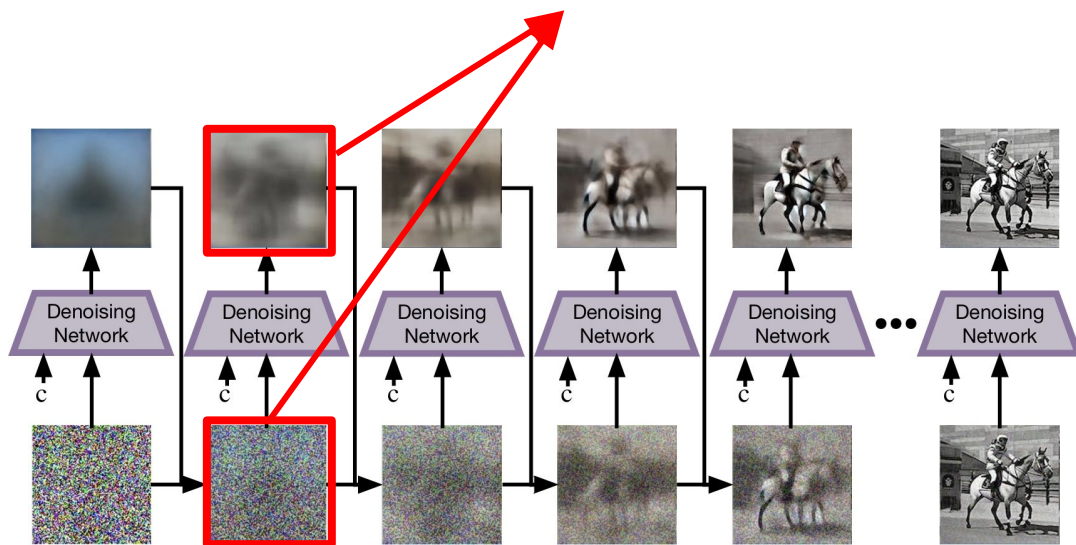
Model Optimization:
distillation and pruning
make the model
smaller and faster.
**They overlooked the
use of systems across
multiple generations.**

[1] Hongkai Zheng, Weili Nie, Arash Vahdat, Kamyar Azizzadenesheli, and Anima Anandkumar. Fast sampling of diffusion models via operator learning. In ICML 2023.

[2] Chenlin Meng, Robin Rombach, Ruiqi Gao, Diederik Kingma, Stefano Ermon, Jonathan Ho, and Tim Salimans. On distillation of guided diffusion models. In CVPR, 2023.

Proposed Caching Technique

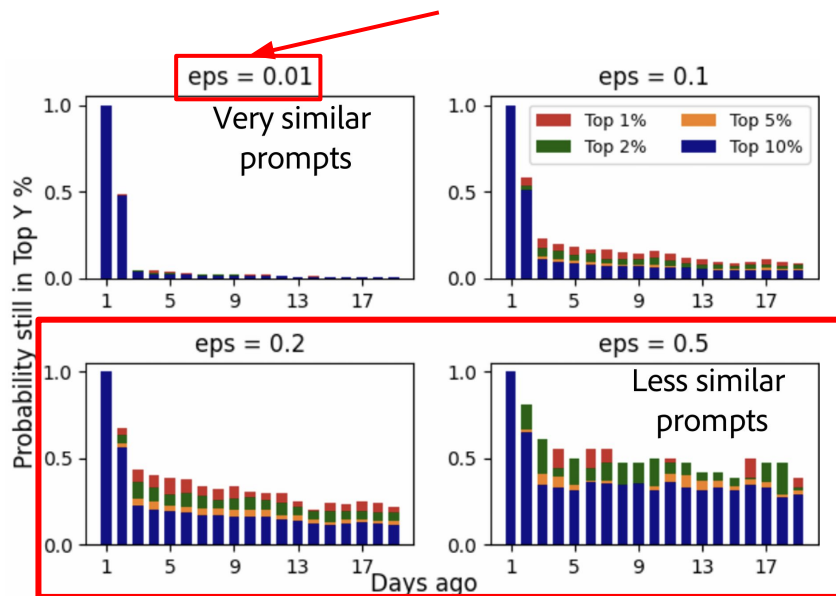
Intermediate states can be cached!



Reduce the generation time by reusing cached intermediate states from similar prior prompts.

Prompt Analysis

max dist between samples within a cluster



Top Prompt Clusters' Popularity Over Days

- Group similar prompts
- Track longevity of popular clusters
- Repeat for different eps

Result: Popular clusters remain “hot” for several days if they are loosely packed.

Conclusion: Overall similarity stays strong over time!

Prompt Analysis

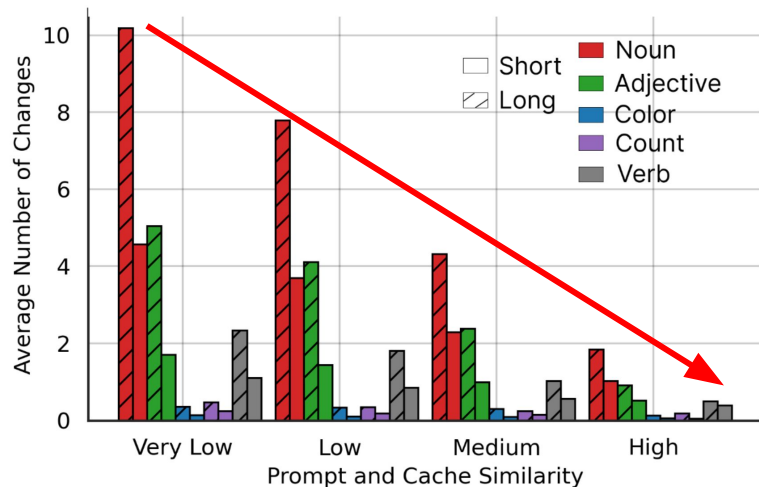


Figure 6: Average number of noun, adjective, color, count, and verb changes across similarity for both short and long prompts

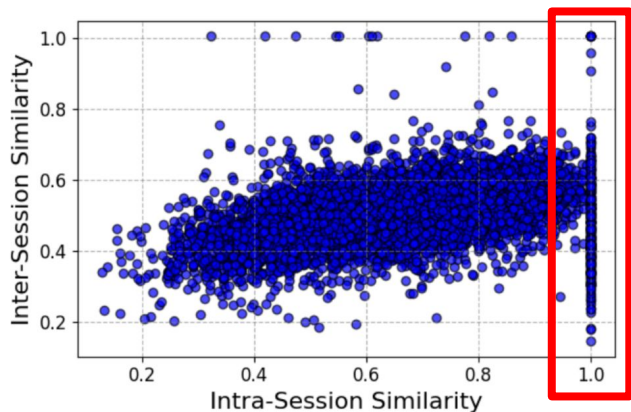
Split prompts based on length
Vary similarity level in pairs of prompts

Track avg # of attribute changes

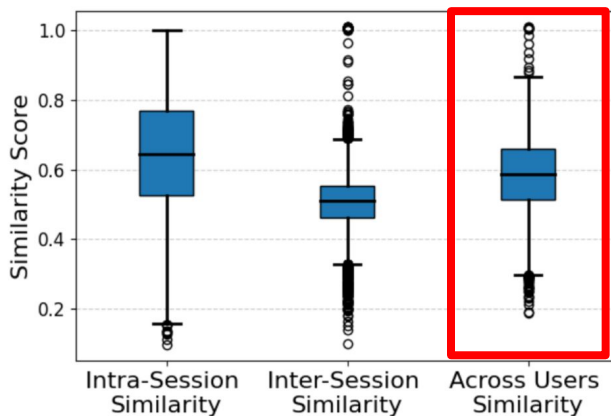
Result: As the cos-sim increases, the average # of changes in attributes decreases.

Conclusion: cos-sim as a good metric to indicate similarity between prompts

Prompt Analysis



(a)



(b)

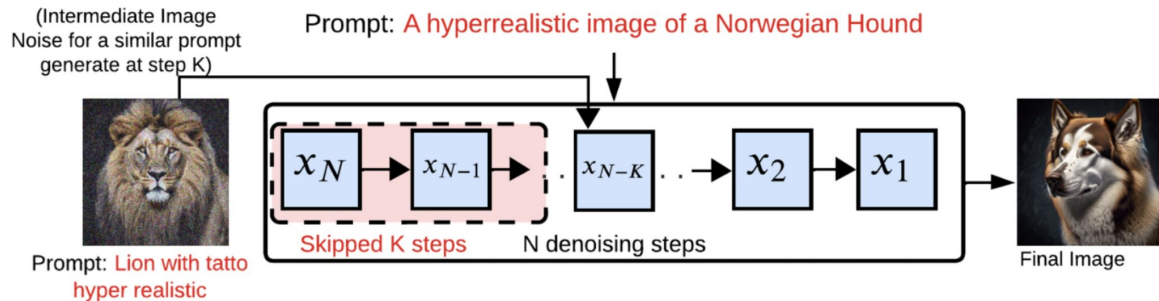
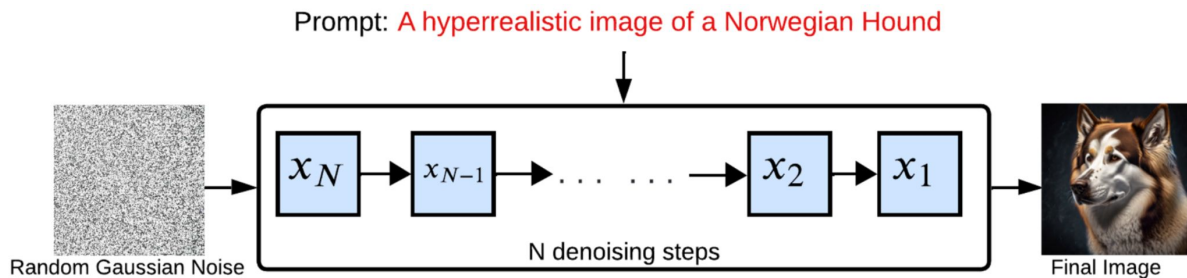
Figure 7: Query prompt similarity (a) for inter v/s intra-session per user (b) range for intra, inter per user and across diff users

High Intra-session Similarity: Users tend to use similar prompts within a session.

High Inter-user Similarity: Different users often request similar prompts.



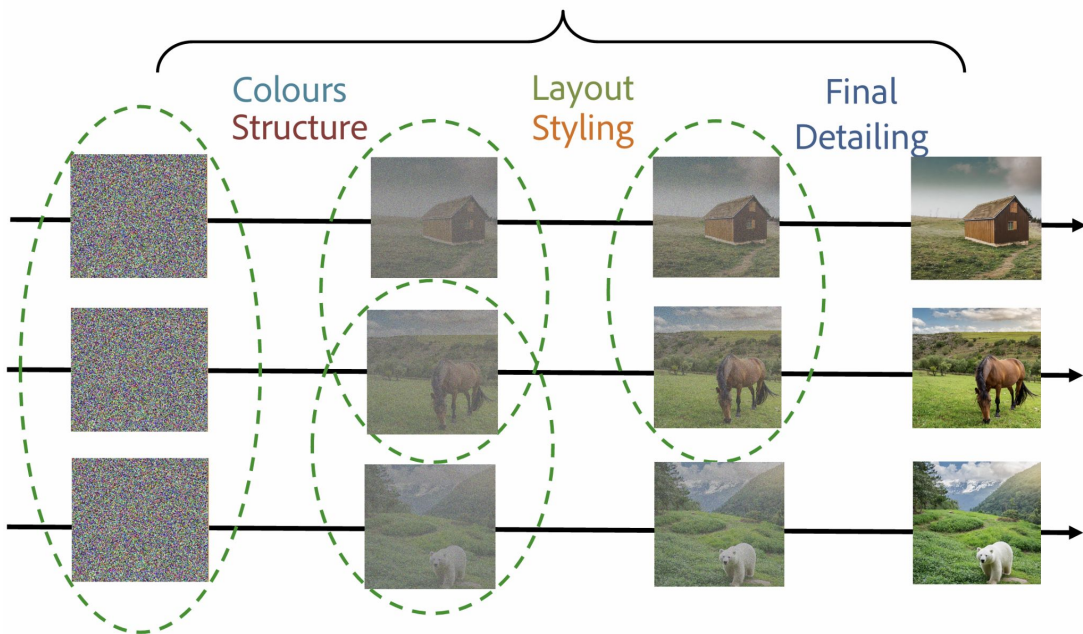
Proposed Approximate Caching





Proposed Approximate Caching

Different aspects of image forms across different steps



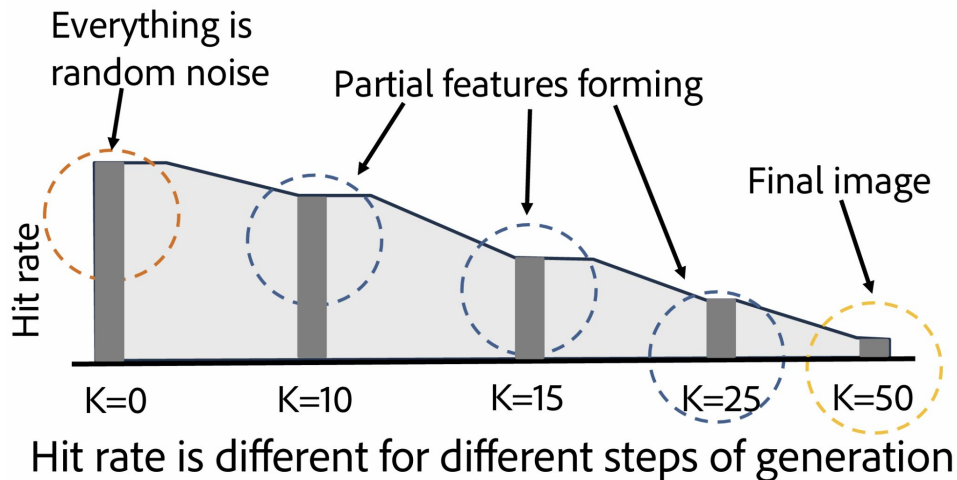
- Different features form gradually
- Cache early intermediate states
- Skip initial steps for future prompts



Proposed Approximate Caching

Opportunity: Cache and generate from intermediate step

$$\text{Maximize compute savings } f_C = h(K) \cdot \frac{K}{N}$$



K: # of skipped steps
N: original # of steps to generate an image
Hit: if an image can be reconditioned
 $h(k)$: Hit rate using $N-K$ steps

Cache Selection

Retrieved prompt:

A **brown** horse **grazing** in a green field



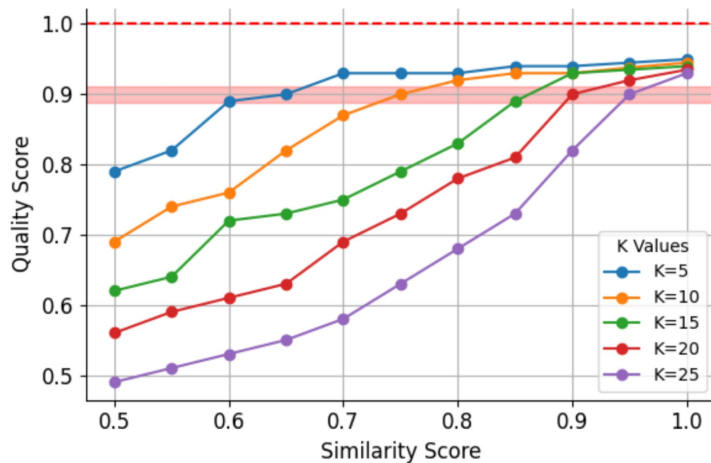
	Worked	Worked	Worked	Worked
	"A white horse grazing in a green field"	"A brown horse running in a green field"	"A brown bear grazing in a green field"	"A brown horse grazing near a river "
				
	K = 10	K = 10	K = 20	K = 20
Did not work				
	K = 15	K = 15	K = 25	K = 25

How to decide K?
Conclusion: More steps can be skipped for a prompt if its cache is more similar

K is Determined by Prompt-Cache Similarity:

Noise from a Brown Horse Transforms into Different Prompts, Limited by K

Cache Selection

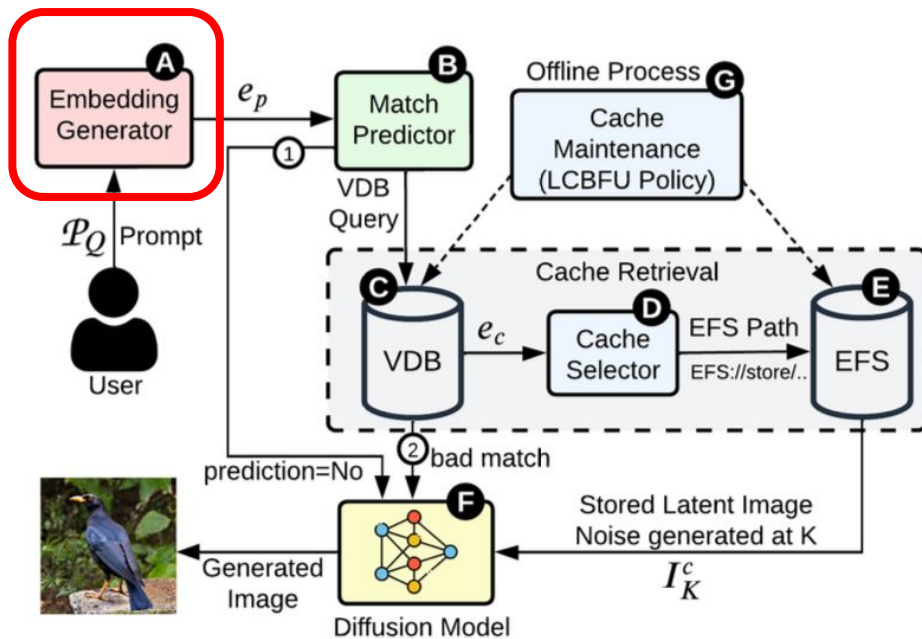


- Generate images for queries at different K values
- Profile image quality for different prompt-cache similarity levels

```
def cache_selector(s):  
    if s > 0.95: k = 25  
    elif s > 0.9: k = 20  
    elif s > 0.85: k =  
        15  
    elif s > 0.75: k =  
        10  
    elif s > 0.65: k = 5  
    else: k = 0  
    return k
```

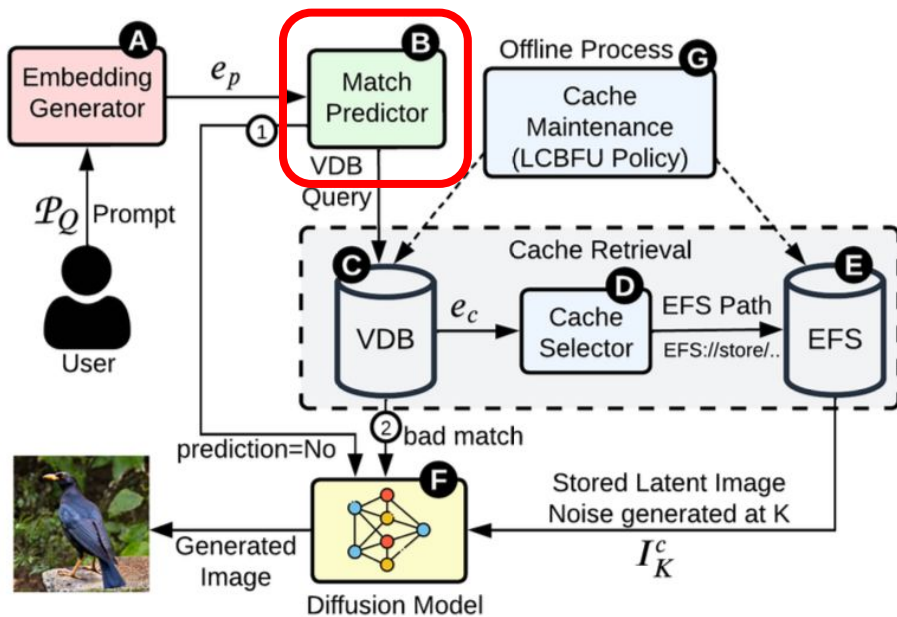
- Map from similarity score to the max K that can be skipped

System Pipeline



A. NIRVANA first computes a 768-dimensional vector CLIP embedding

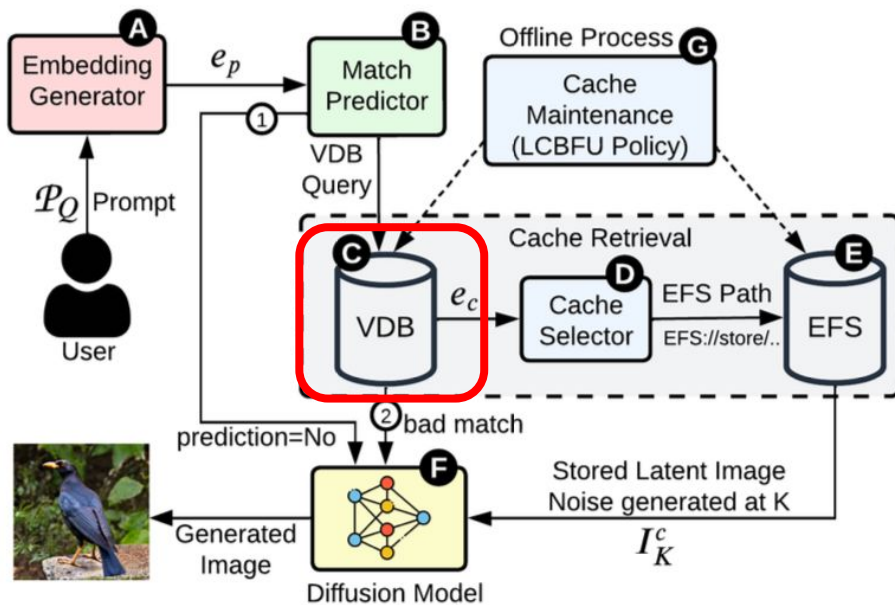
System Pipeline



B. Match Predictor predicts if an embedding close enough to e_p is likely to be present in the VDB.

Match predictor is a One-Class SVM trained with embedding in VDB.

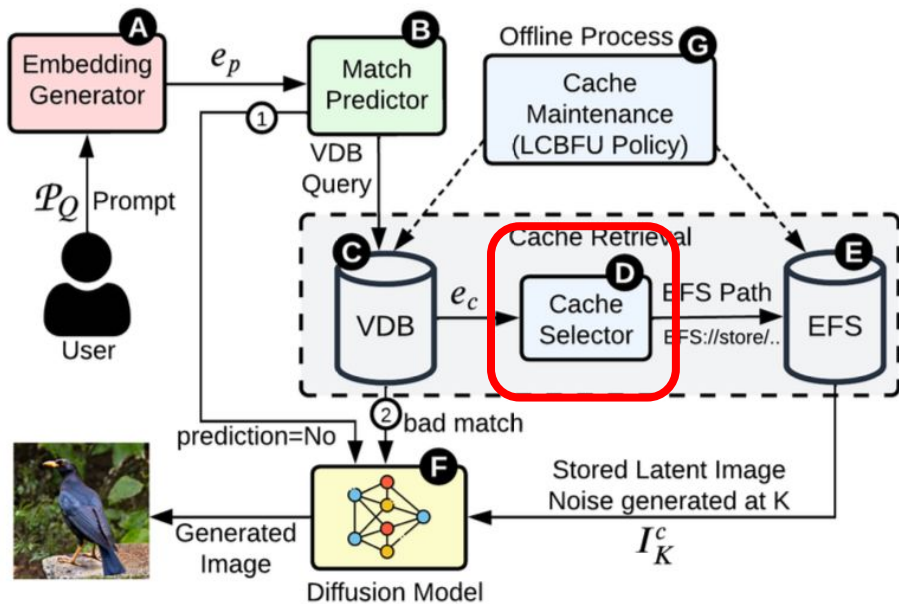
System Pipeline



C. NIRVANA stores the embedding of the historical prompts in a vector-database.

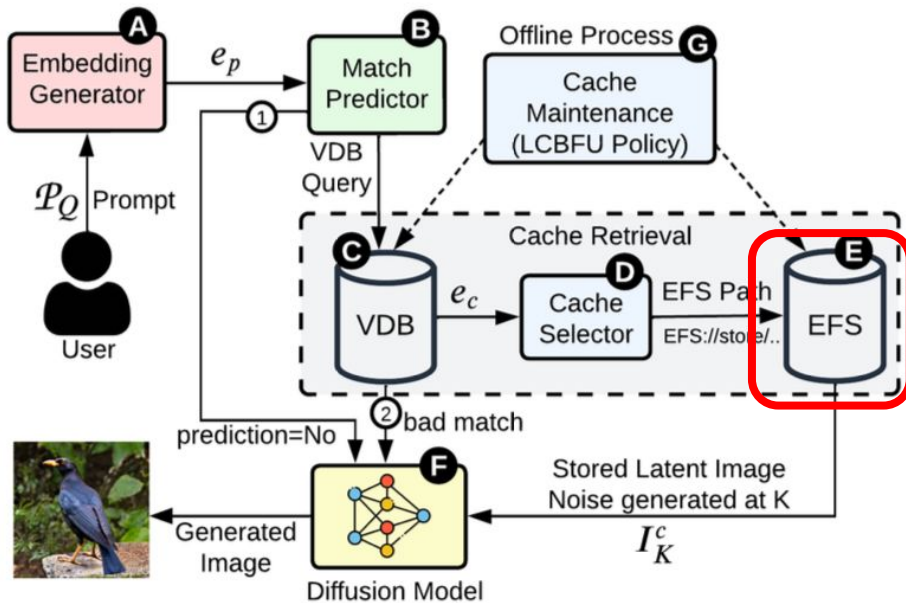
Vector-database deployed on AWS EC2 and connected to system through sub-network.

System Pipeline



D. Use the cache selection mechanism as mentioned before to determine the layer of Latent noise to retrieve.

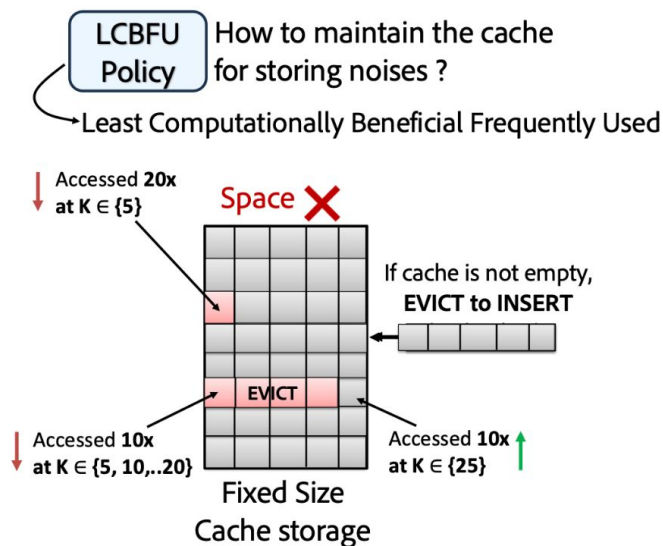
System Pipeline



E. The cache for latent image noise of different steps is stored in EFS.

Nirvana uses AWS elastic file service.

LCBFU Policy



Take into account both the frequency a cache block is accessed and the potential saving (the number of step it can skip).

Eviction Policy?

Least Computationally Beneficial Cache (LCBFU)
Score = K (potential savings) * f (frequency of use)



Evaluation

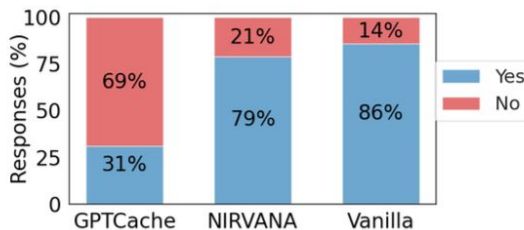
Use real prompt traces from
production with two dataset:
System-X and Diffusion DB

Baseline: GPT-cache (no
recomputation just pure cache),
Small Model, etc

Evaluation - Quality

Evaluate Nirvana quality using FID, CLIP and PickScore.

Conduct a user study with 60 participants.



Dataset	Models	Quality		
		FID ↓	CLIP Score ↑	PickScore ↑
DiffusionDB*	GPT-CACHE	7.98	25.84	19.04
	PINECONE	10.92	24.83	18.92
	CRS	8.43	24.05	18.84
	SMALLMODEL	11.14	25.64	18.65
	NIRVANA – w/oMP	4.94	28.65	20.35
	NIRVANA	4.68	28.81	20.41
	VANILLA	6.12-6.92	30.28	20.86
SYSTEM-X	GPT-CACHE	8.15	26.32	19.11
	PINECONE	10.12	24.43	18.83
	CRS	8.38	23.81	18.78
	SMALLMODEL	11.35	25.91	18.92
	NIRVANA – w/oMP	4.48	28.94	20.31
	NIRVANA	4.15	29.12	20.38
	VANILLA	5.42-6.12	30.4	20.71



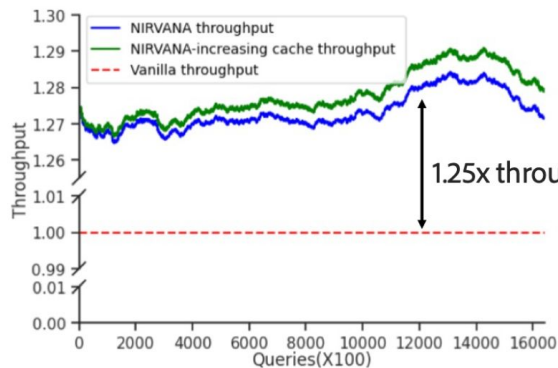
Evaluation - Latency

	Latency(s)	90 th / median	95 th / median	99 th / median
<i>GPTCache</i>	2.8	29.52	29.64	29.75
NIRVANA	6.9	1.20	1.21	1.21
VANILLA	8.6	1.01	1.02	1.02

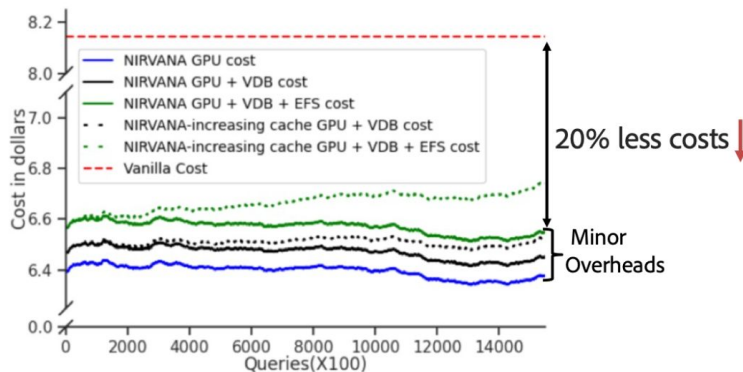
The latency for Nirvana is 20% lower than Vanilla while maintaining similar quality.

Evaluation - Cost and Throughput

The throughput of Nirvana System is 25% higher than vanilla while the cost is 20% lower.



Throughput attained by NIRVANA
w.r.t. standard deployment



Cost analysis of deploying NIRVANA
w.r.t. standard deployment



Evaluation - LCBFU Policy

Cache Size	#noises in cache	FIFO	LRU	LFU	LCBFU
1GB	1500	0.58	0.65	0.64	0.65
10GB	15000	0.68	0.77	0.78	0.77
100GB	150000	0.74	0.85	0.83	0.82
1000GB	1500000	0.83	0.95	0.94	0.93

We evaluate the cache hit rate and compute saving by comparing it with traditional eviction policy.

(a) Hit rate

Cache Size	#noises in cache	FIFO	LRU	LFU	LCBFU
1GB	1500	0.11	0.12	0.12	0.12
10GB	15000	0.13	0.14	0.14	0.15
100GB	150000	0.14	0.16	0.16	0.18
1000GB	1500000	0.17	0.20	0.19	0.23

(b) % Compute savings



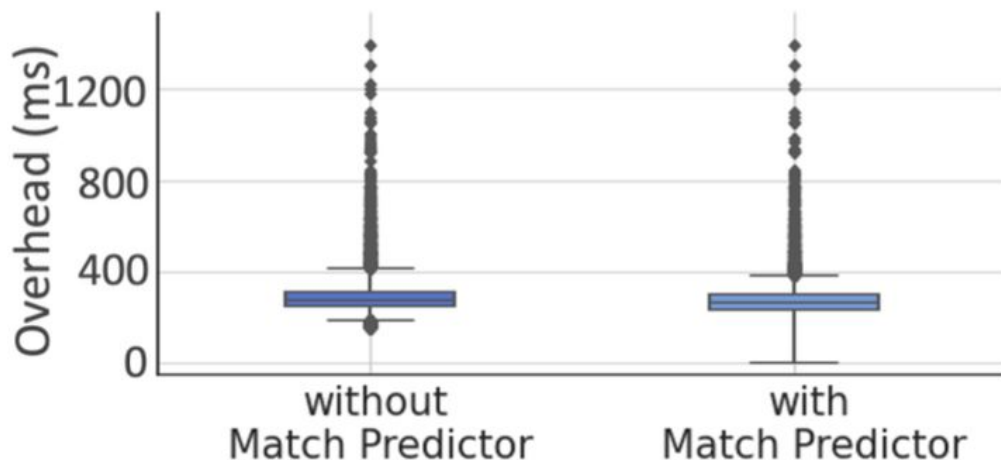
Sensitivity Analysis - Prompt embedding

Embedding	Prompt/Cache	FID↓	CLIPScore↑	HitRate↑	ComputeSavings↑
<i>BERT</i>	Query & Cache	4.53	28.94	0.77	0.16
<i>CLIPImage</i>	Only Cache	4.85	28.49	0.84	0.18
<i>CLIP</i>	Query & Cache	4.94	28.65	0.93	0.23

We experiment with different text embedding generator on their quality, hit rate and compute savings.



Sensitivity Analysis - Match Prediction



We assess the effectiveness of match-predictor by measuring overhead. Lower minimum overhead.



Comparison

Feature	Cornserve: Any-to-Any Serving	NIRVANA: Approximate Caching
Key Idea	Solves resource allocation by abstracting GPUs into cells to balances heterogeneous components	Reuses intermediate noise states from similar prior prompts to reduce compute



Limitations

NIRVANA

1. Domain Specific: Only applicable to iterative models (diffusion)
2. Cache Dependencies: Effectiveness relies heavily on prompt similarity in the workload



Limitations

Cornserve

1. Complexity: Requires users to define explicit computation paths.
2. The overhead of the framework is not thoroughly understood
3. KVCache bottleneck limits optimization in text-heavy workloads
4. Replica-level scheduling and placement is not clearly explained.



Limitations

Cornserve

1. Complexity: Requires model developers to define explicit computation paths.
2. KVCache bottleneck limits optimization in text-heavy workloads



Future Direction

Nirvana:

Integrate Nirvana's caching strategy with scheduling optimization:
shown in Tetri Serve

Explore if similar mechanism can be used in other generator model



Future Direction

CornServe

- Evaluating CornServe on workloads that change overtime.
- Scaling CornServe towards larger clusters: May reveal opportunities such as incremental reconfiguration.
- Performing memory-aware optimizations to unlock potential in decode heavy workloads.



References

<https://www.usenix.org/system/files/nsdi24-agarwal-shubham.pdf>

https://www.usenix.org/system/files/nsdi24_slides-agarwal_shubham.pdf

<https://journal.everypixel.com/ai-image-statistics>

https://www.cs.cornell.edu/courses/cs4782/2025sp/slides/pdf/week9_2_slides.pdf

<https://overrideturing.github.io/2024/11/04/diffusion%E5%8F%91%E5%B1%95%E8%84%89%E7%BB%9C/index.html>